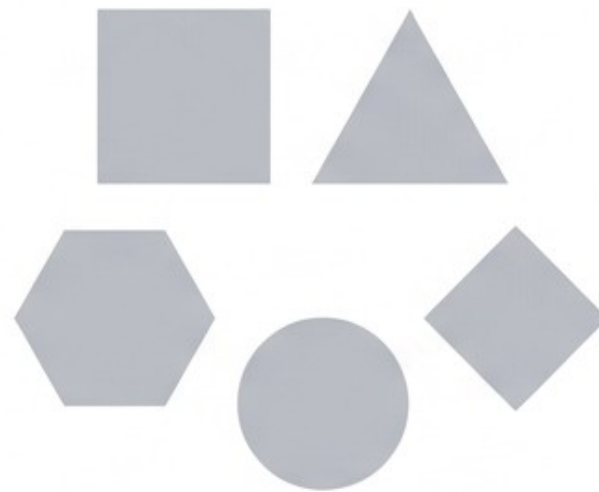
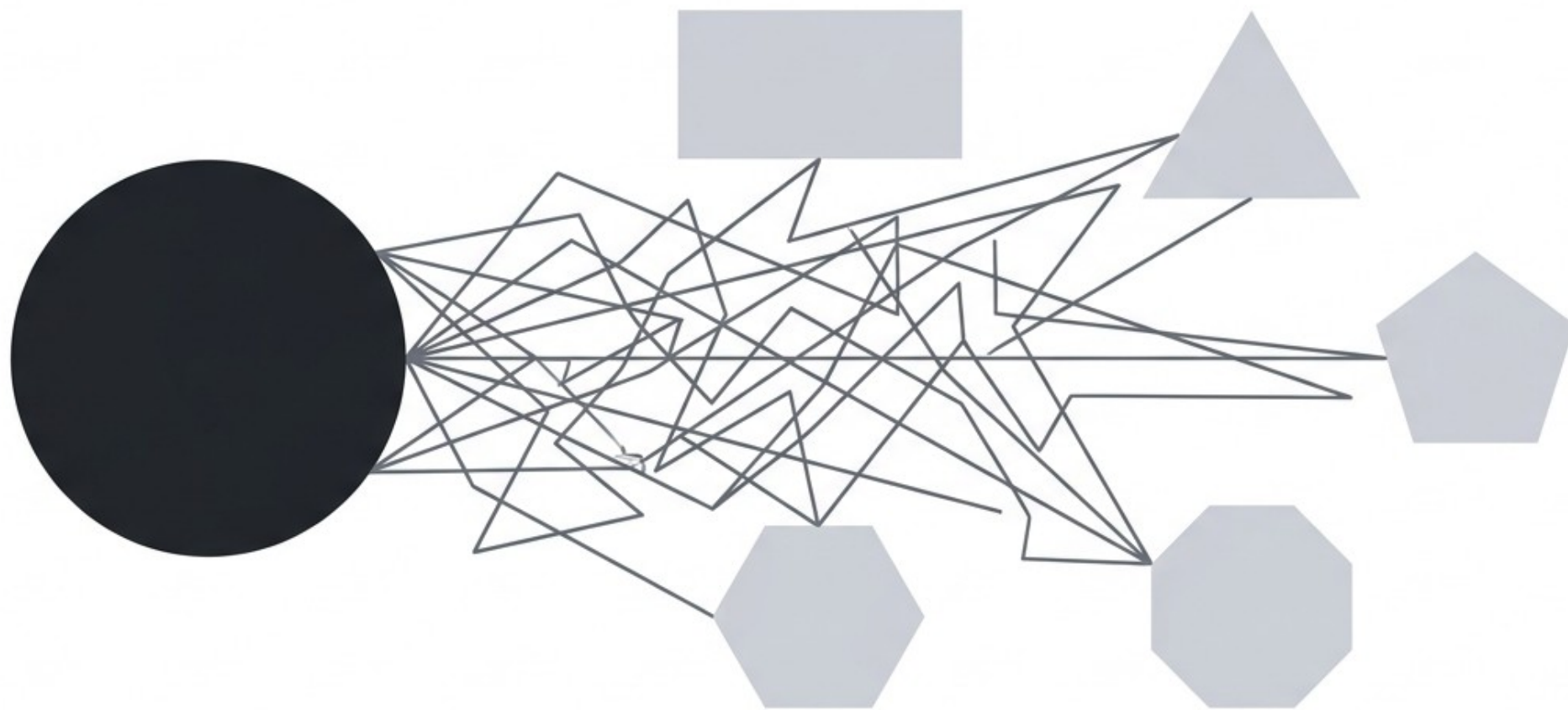


Model Context Protocol

The open standard for giving AI access to the real world.



AI is smart. Connecting it to your tools is a mess.



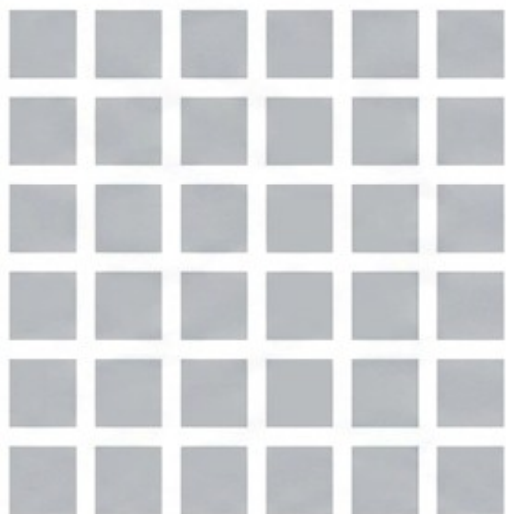
Every API behaves differently.

Every integration requires custom glue code.

Every model update breaks the connection.

The API Mismatch

TRADITIONAL APIs



Designed for Programs.
Require precise, deterministic requests from systems that already know exactly what they want.

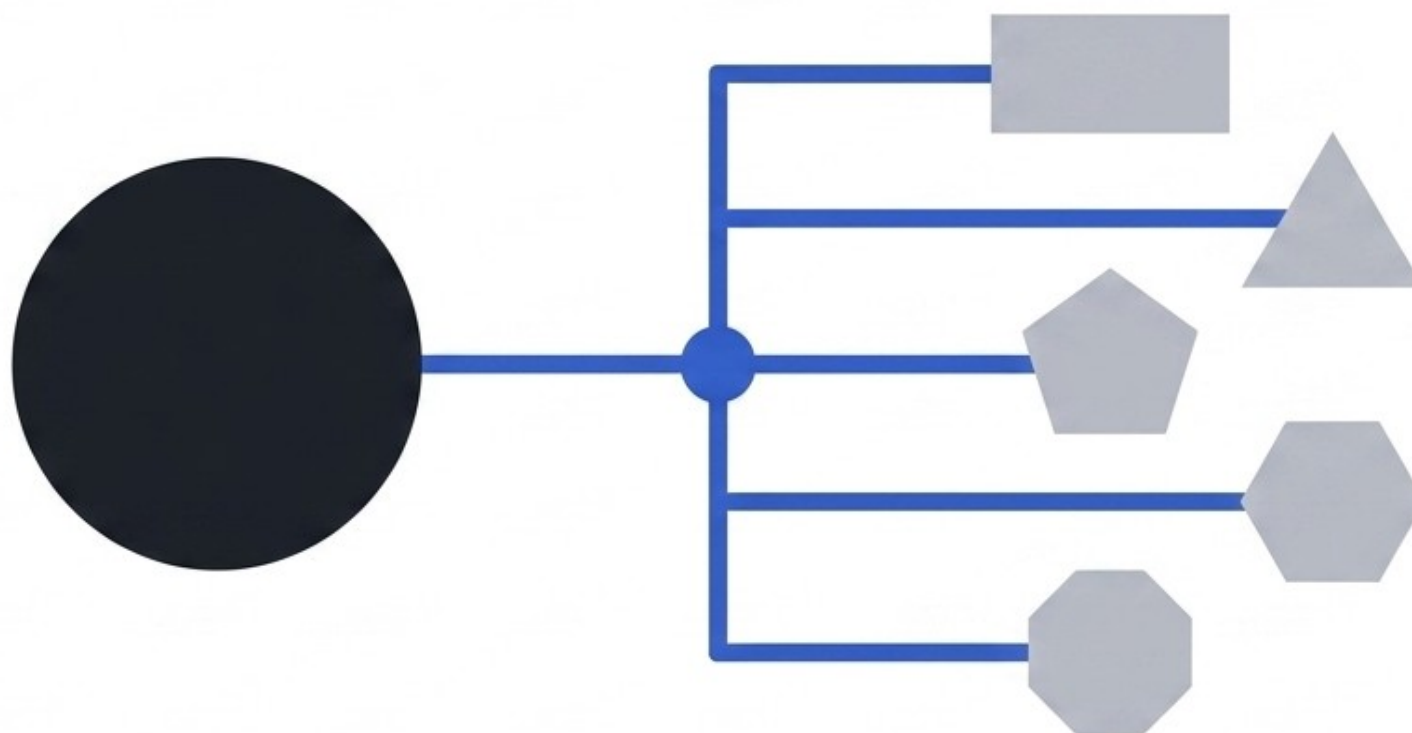
AI MODELS



Designed to Reason. Function probabilistically. They need to ask questions, explore, and clarify before they know what to do.

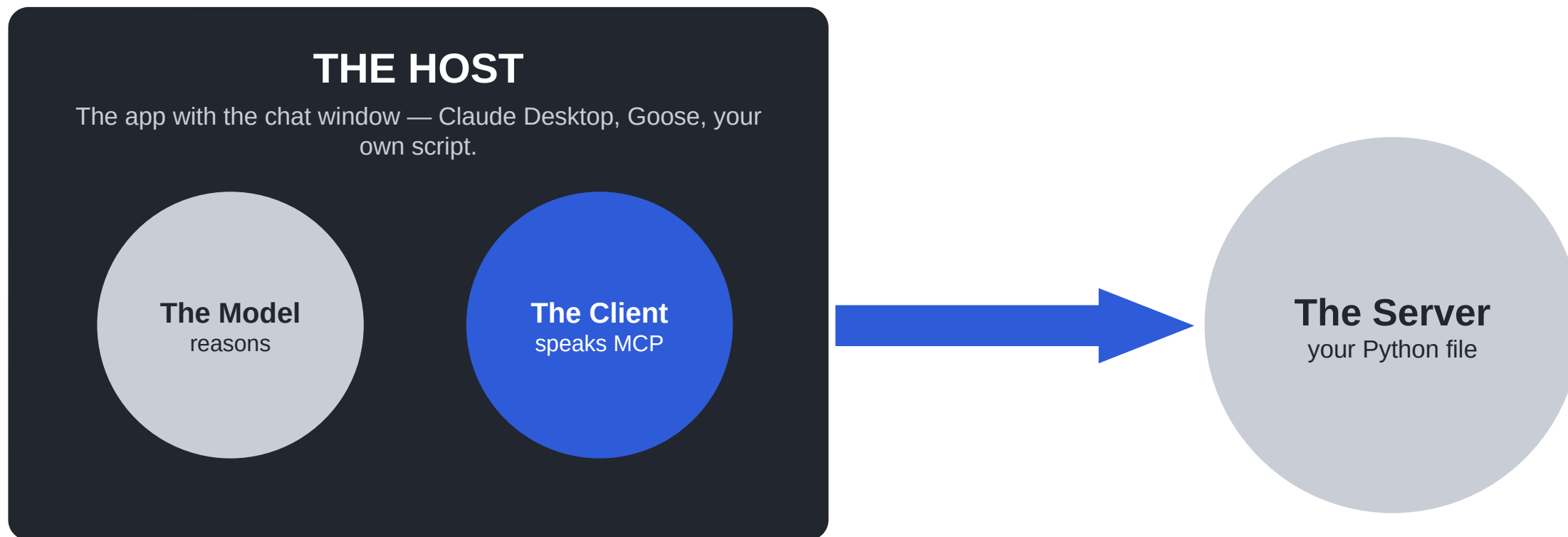
A shared language between models and systems

The Model Context Protocol (MCP) bridges the gap, allowing AI to discover and interact with its environment dynamically.



The architecture: three roles, not two.

Host, Client, Server — and the language model is none of them.

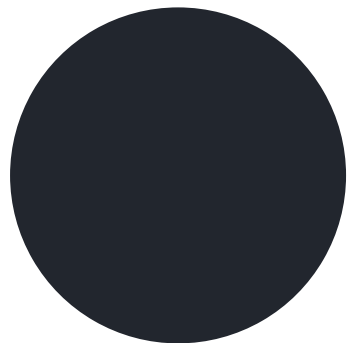


The model is NOT the client. The model reasons; the client is the plumbing inside the host that speaks the protocol.

The server exposes tools, resources, and data — a database, the file system, a web service.

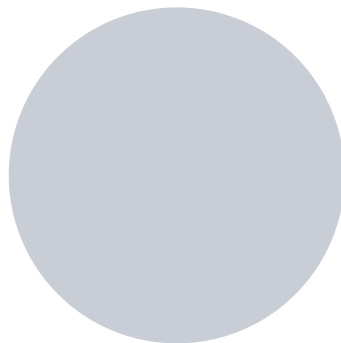
What the AI can actually see and use

Every MCP server can offer three primitives. Three — not four.



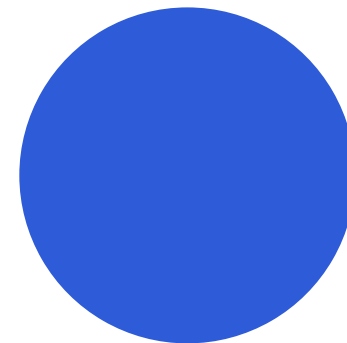
Tools

Actions the model can invoke
(e.g., Search Database, Send Email).



Resources

Pieces of data and state the host
reads
(e.g., text documents, database rows).

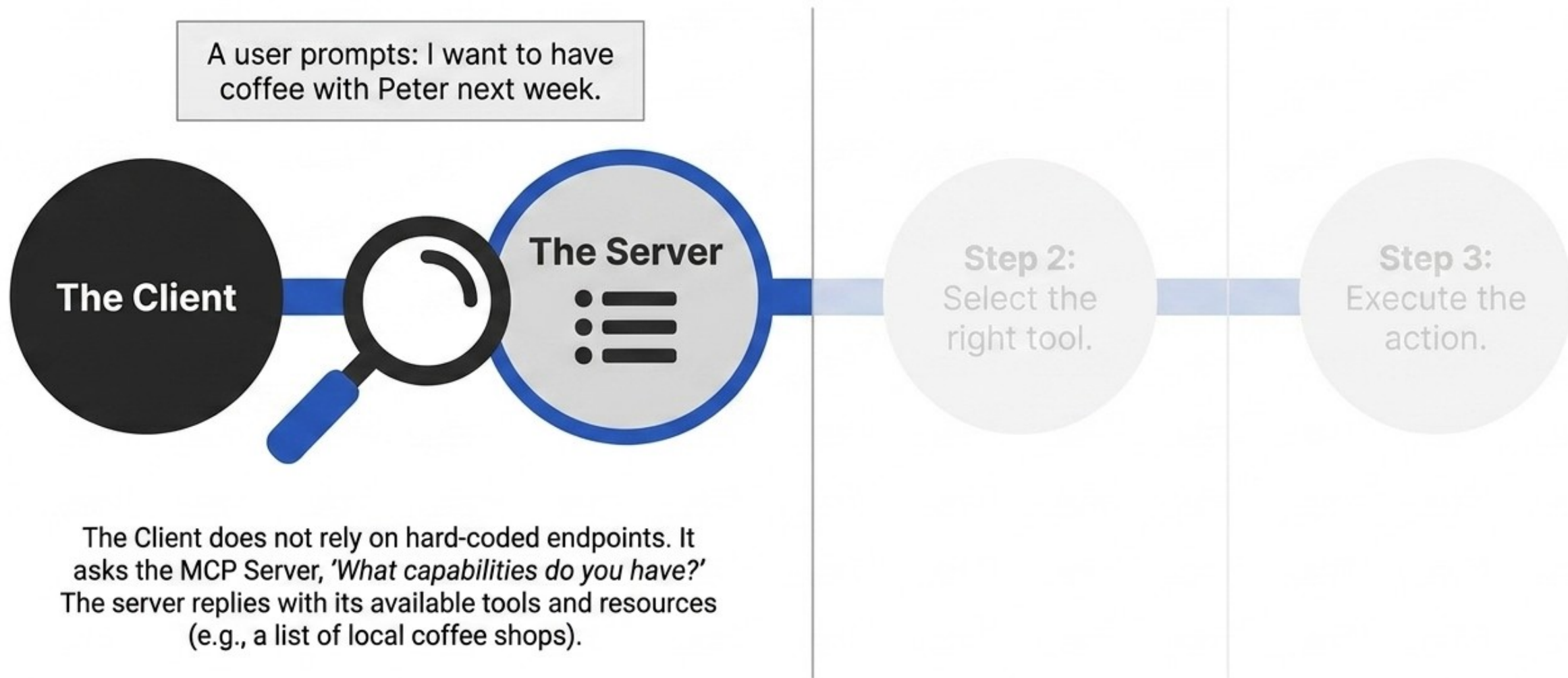


Prompts

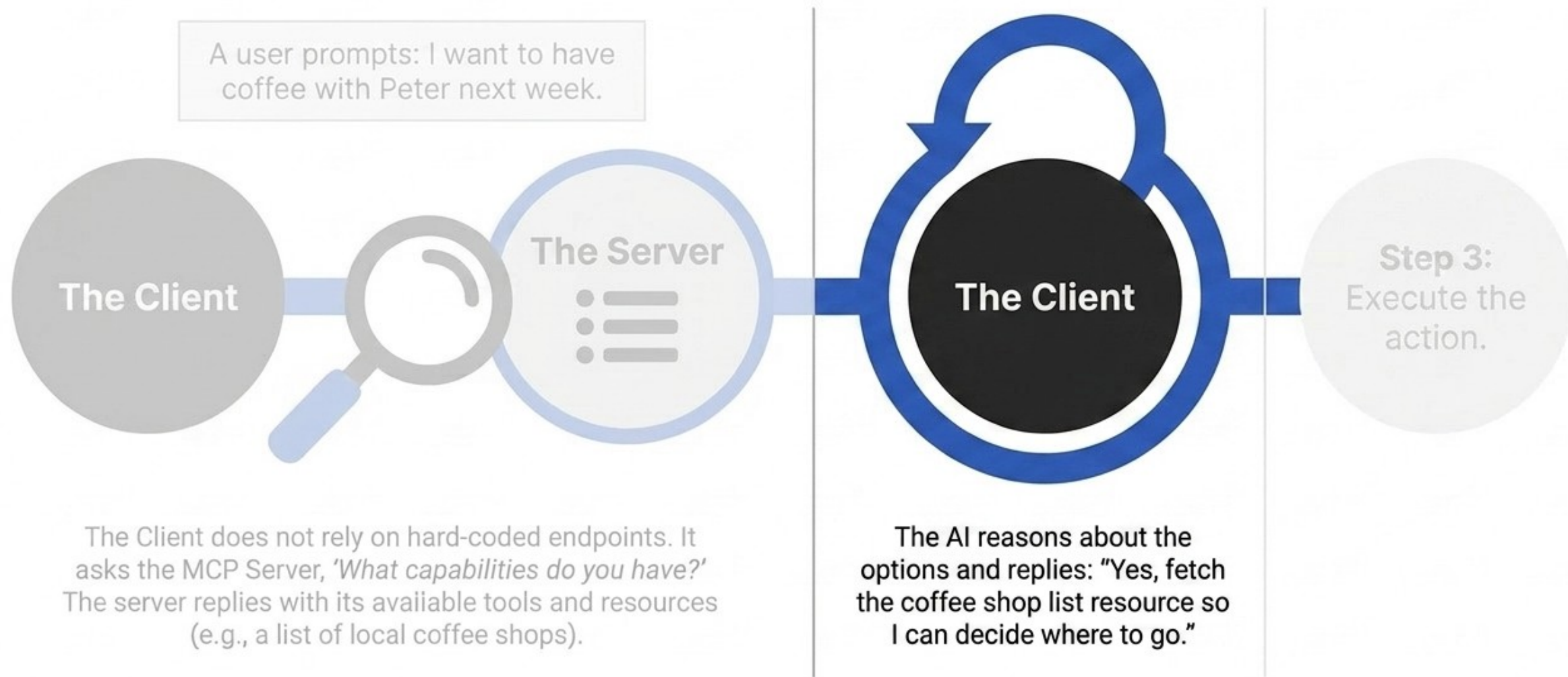
Reusable templates the user
triggers
for specific tasks.

Where did “context” go? “Context” is not a fourth primitive — it's the RESULT. These three are how context reaches the model's context window.

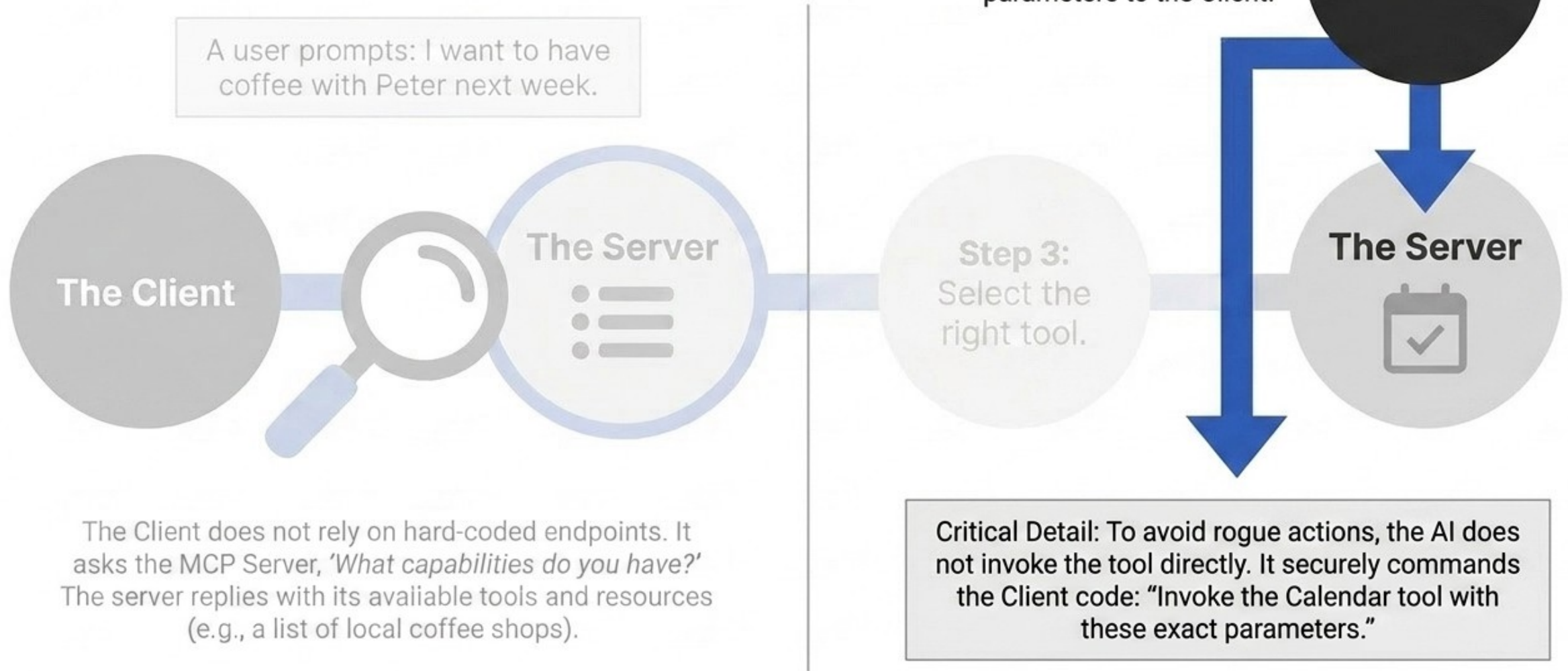
Step 1: Discover capabilities on the fly.



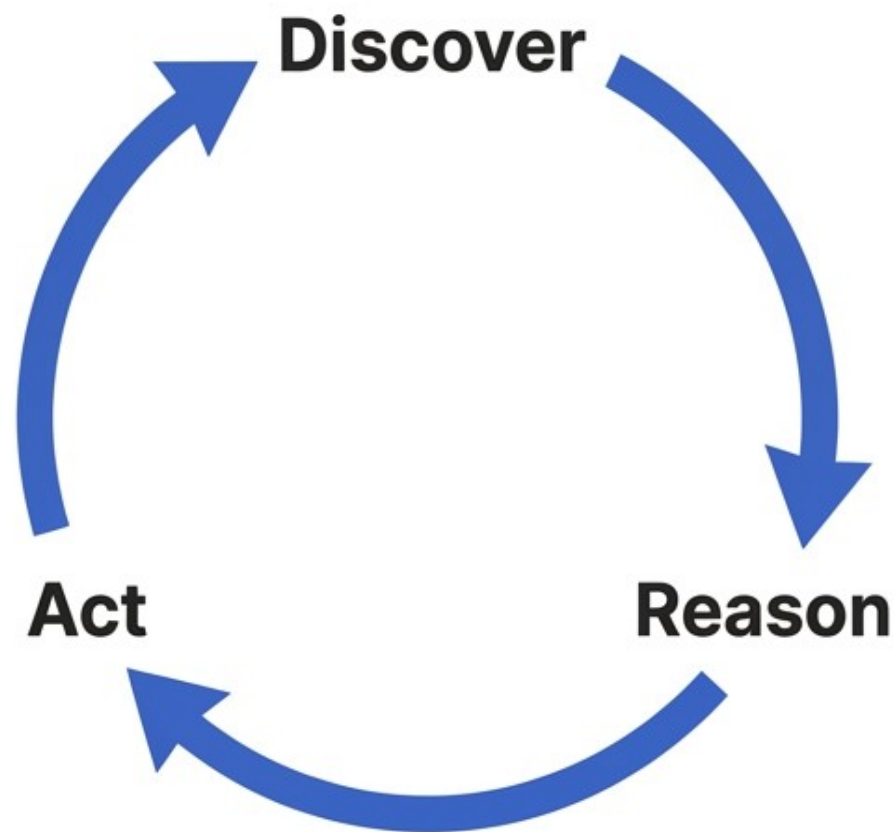
Step 2: Reason through the options.



Step 3: Take structured, safe action.



The dynamic interaction loop



By standardizing Discovery, Reasoning, and Action, MCP allows agents to figure out their environment dynamically. No pre-programming required.

Write the connection once. Use it anywhere.

THE OLD WAY



Custom integration means managing authentication, rate limits, and edge cases for every single service.

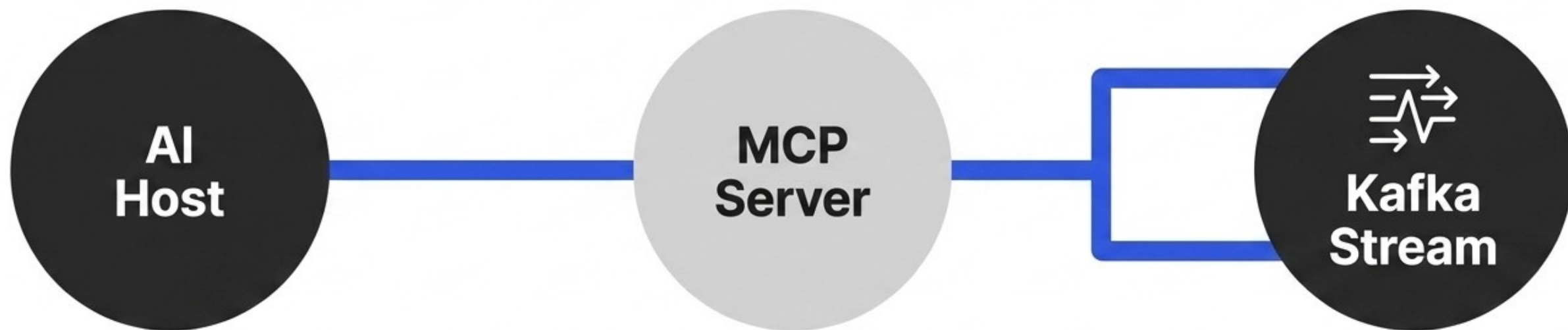
THE MCP WAY



The protocol ensures the structure is perfectly matched to what the tool expects, universally.

Boundless composability.

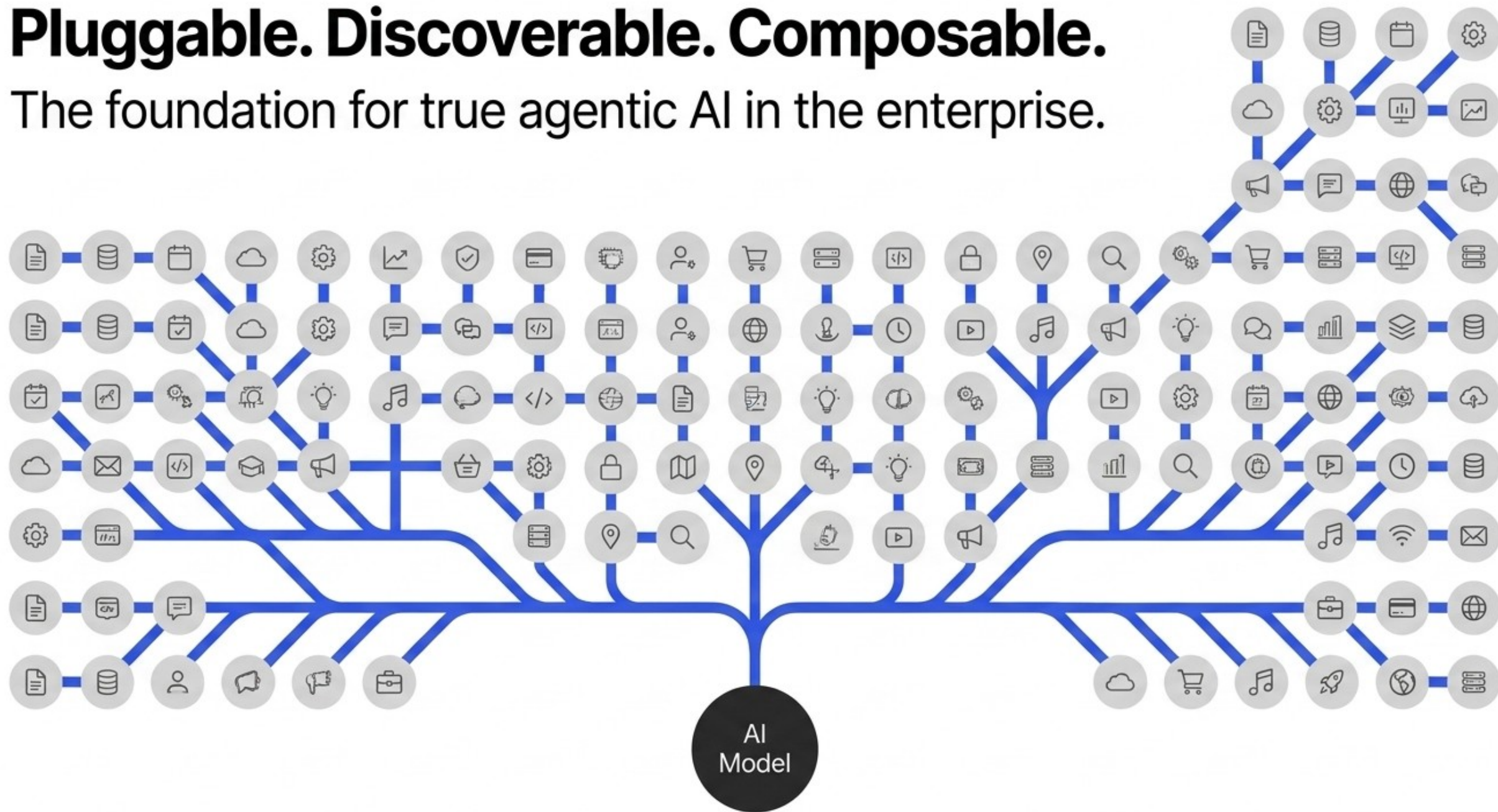
Servers can act as clients.



An MCP server doesn't just talk to local files. It can act as a client to another server—bringing vast, real-time enterprise data into the agent's context without writing new core integrations.

Pluggable. Discoverable. Composable.

The foundation for true agentic AI in the enterprise.



Enough theory.

Let's build a server.

One Python file. Two tools. A real product connected by the end of the hour.

First: watch three questions fail.

Live demo — a local model, no MCP. Same root cause every time.

1 — Filesystem “What is written in the file notes.txt in my current directory?”



2 — Live web data “What is the weather in Tel Aviv right now? Exact temperature.”



3 — Database “Show me all products in our workshop database.”



```
python demo_act1_failures.py
```

The LLM is trapped inside a box. MCP is the door.

A server in two lines.

FastMCP — the framework. The name shows up in every client that connects.

```
from mcp.server.fastmcp import FastMCP  
  
mcp = FastMCP("Workshop")
```

```
$ python server_starter.py  
# server running — zero tools registered
```

Empty server. No tools. Nothing the LLM can call — yet. We're going to change that.

The decorator is the whole trick.

Same function. One line on top registers it as something the LLM can call.

WITHOUT — just a function

```
def get_weather(city: str) -> str:  
    return "sunny"
```

WITH — an MCP tool

```
@mcp.tool()  
def get_weather(city: str) -> str:  
    return "sunny"
```

Seen this before? Flask: `@app.route('/weather')` tells the framework “this function handles this URL.”

MCP: `@mcp.tool()` tells FastMCP “this function is a tool the LLM can call.” The pattern of every tool you'll ever write:
decorator on top · clear docstring · function body · return a string.

Tool 1: read the machine.

Failure #1, fixed. And the docstring is not documentation — it's a prompt.

```
@ m cp.tool()
def read_file(filename: str) -> str:
    """Read any text file from the current directory
    and return its full contents."""
    path = Path(__file__).parent / filename
    if not path.exists():
        return f'File not found: {filename}'
    return path.read_text(encoding="utf-8")
```

The docstring IS the prompt.

This is what the LLM reads to decide whether to call your tool. Vague docstring → wrong calls. Precise docstring → good decisions.

Docstrings are prompt engineering.

```
$ python client.py
> "What's in notes.txt?" → it works.
```


Tool 2: reach the live world.

Failure #2, fixed. Free API, no key, no signup — and the data is from right now.

```
@mcp.tool()
def get_weather(city: str) -> str:
    """Get the current weather conditions and
    temperature for any city in the world."""
    import requests
    try:
        r = requests.get(f'https://wttr.in/{city}?format=3', timeout=5)
        r.raise_for_status()
        return r.text.strip()
    except Exception as e:
        return f'Weather unavailable: {e}'
```

Output:
Tel Aviv: + 28°C

Ask the room: why is import requests INSIDE the function? (Isolation — a missing package breaks one tool, not the whole server.)

One file. Two transports. Zero config.

HTTP when you run it from a terminal. STDIO when a product runs it as a subprocess.

```
if __name__ == "__main__":  
    if sys.stdin.isatty():  
        mcp.run(transport="streamable-http") # you, in a terminal  
    else:  
        mcp.run() # STDIO - Goose / Claude Desktop
```

STDIO

Host launches your server as a subprocess. The desktop story.

Streamable HTTP

Server listens on a port. The remote / cloud story. (SSE is deprecated.)

DO NOW — your turn (25 min)

Build a tool nobody else in the room has.

1

Pick a capability — A free API (wttr.in, Open-Meteo, Wikipedia...) or something on your machine.

2

Write the tool — `@mcp.tool()` · docstring that reads like a prompt · body · return a string.

3

Restart + test — `python server_starter.py` → `python client.py` — ask a question that needs YOUR tool.

4

Break it on purpose — Blank the docstring. Watch the LLM stop calling your tool. Now you believe it.

Finish early? Add a second tool that takes TWO parameters.

You built the door.

One Python file. The same pattern, forever:

```
@ m cp .tool()
```

```
docstring = prompt
```

```
do one thing
```

```
return a string
```

Today: your model read a file and fetched live weather — the two failures from minute one, fixed by ~40 lines of Python.

Tomorrow: we flip sides — you write the CLIENT, and connect real products (Goose, Claude Desktop) to the server you just built.

Remember the two corrections: the client is NOT the model (it lives inside the host) · MCP has THREE primitives — tools, resources, prompts.